Session 3: Developer Foundations & Your First Agent

Session 3 - Developer Foundations & Introduction to Agentic AI

Welcome back! In the last two sessions, we looked at transformers, attention, embeddings, tokenization, scaling laws, alignment. You now have some understanding about WHY LLMs work.

Today we take a very important step. We go from understanding the LLMs to converting them into Agents.

Today, you will have built your first AI agent. But before we get there, we need to make sure you have the right tools in your hands. We'll sprinkle Python and other code snippets (yes in 2026!!)

Part 1: Python That Helps You Debug Agentic Systems

Let's be honest!

Claude Code, Cursor, Copilot, these tools are incredible. I use them. You should use them.

But they've created a dangerous illusion.

People are shipping code they cannot read. They're building systems they cannot debug. And when something breaks at 2 AM, and it will, they stare at a stack trace and have no idea what to do. (specially when their quota ends 🤣🤣😂 and they have to submit assignments)

In this course, you're going to build systems where an LLM makes decisions in a loop, calls functions, gets results, and makes more decisions. When this goes wrong, and trust me, it goes wrong in the most creative ways, you can't just re-

prompt your Copilot. You need to pause execution, inspect the state, trace what happened, and understand WHY your agent chose to call the wrong tool or got stuck in an infinite loop.

These are not random Python tips. These are survival tools for building agentic systems.

code.interact(), Your Emergency Inspection Window

The `code.interact()` function is a hidden gem in Python that allows you to drop into an interactive shell at any point in your script, inspect or modify variables, and then continue execution (if used correctly).

Let me show you the simplest version first:

```
import code

x = 42
y = "Hello TSAI"
```

```
# Drop into interactive shell
code.interact(local=locals())

print("This will execute if you use Ctrl+D")
```

code.interact() drops you into a live Python shell where you can inspect and modify any variable in scope

What happens here?

1. The script runs until `code.interact(local=locals())`.
2. The program pauses and enters an interactive shell where you can inspect and modify `x` and `y`.
3. Exiting with `exit()` will terminate the script immediately, meaning **the next print statement will NOT run**.
4. Exiting with `Ctrl+D` will NOT terminate the script immediately, meaning **the next print statement WILL run**.

But here's why this actually matters for you. Imagine you're building an agent and it calls the wrong tool.

You want to freeze execution right before the tool call and see what the LLM actually returned:

```python
import code
import json

def agent_loop(user_query):
    conversation = [{"role": "user", "content": user_query}]

    while True:
        llm_response = call_llm(conversation)

        # FREEZE HERE - inspect what the LLM actually said
        code.interact(local=locals())

        # Was it a tool call or final answer?
        parsed = json.loads(llm_response)
        if parsed.get("tool_name"):
            result = call_tool(parsed["tool_name"], parsed["arguments"])
            conversation.append({"role": "tool", "content": result})
        else:
            return parsed["answer"]
```

When you drop into that shell, you can type `llm_response` and see EXACTLY what the model returned. You can type `conversation` and see the full history. You can type `len(conversation)` to check how many turns deep you are. This is how you debug agents.

But remember, `code.interact()` is not a debugger. For that, we need something more powerful.

pdb, Python's Built-In Debugger

The `pdb` module is Python's built-in interactive debugger. It allows you to **pause execution, inspect variables, step through code line-by-line, and set breakpoints**, all without modifying your script permanently.

```
from pdb import set_trace

x = 10
set_trace()  # debugger starts here
y = 20
print(x + y)
```

pdb pauses at set_trace() and shows you exactly which line will execute next

What happens here?

1. The script runs, pauses at `set_trace()` and drops you into a debugging prompt (`(Pdb)`).

2. You can **inspect variables, modify them and step through the code** interactively.
3. Type `c` (continue) to resume execution.

Essential `pdb` commands:

Command	Description
`c` or `continue`	Resume execution until the next breakpoint
`h` or `help`	Show available commands
`s` or `step`	Execute the next line and step INTO functions
`n` or `next`	Execute the next line but DON'T step into functions
`p variable`	Print the value of a variable (`p x` or just `x`)
`pp variable`	Pretty-print variable (`pp my_dict`)
`q` or `quit`	Quit the debugger and stop execution
`b 10`	Set a breakpoint at line 10

Command	Description
`cl`	Clear all breakpoints

Step-by-step debugging example:

```
def add(a, b):
    result = a + b
    return result

x = 5
y = 15
from pdb import set_trace
set_trace()  # Debugger starts here

z = add(x, y)
print("Result:", z)
```

pdb lets you step INTO functions, inspect variables at every point, and set breakpoints, this is how you trace execution through complex agent code

The modern way: In Python 3.7+, you can just write `breakpoint()` instead of importing pdb:

```
x = 10
breakpoint()  # same as from pdb import set_trace; set_trace()
y = 20
print(x + y)
```

Why this matters for agents: When your agent loop runs 5 iterations and produces garbage on iteration 3, you set a breakpoint inside the loop, step through iteration by iteration, and see exactly where the reasoning went wrong. You can inspect the conversation history at each turn. You can see what tool was called and what it returned. This is not optional knowledge, this is how professionals debug.

Comparison: `pdb` vs `code.interact()`

Feature	`pdb` Debugger	`code.interact()` Interactive Shell
Pause Execution	Yes (step-by-step)	Yes (interactive shell)
Step Through Code?	Yes	No
Inspect Variables?	Yes (`p var`)	Yes
Modify Variables?	Yes	Yes
Continue Execution?	Yes (`c`)	No (unless in a function)
Set Breakpoints?	Yes (`b line#`)	No
Best Use Case?	**Debugging** (tracing execution)	**Exploring** variables at a point

Python async and await, Because LLMs Are Slow

Python's asyncio module allows you to write **asynchronous, non-blocking code**, making your program efficient, especially for I/O heavy tasks like web scraping, database queries, APIs, and, most importantly for us, **waiting for LLMs to respond**.

Think about it. A single LLM API call takes 1-5 seconds. If your agent needs to call the LLM 5 times in a row, that's 5-25 seconds of your program doing absolutely nothing except waiting. And if you're running multiple agents? Or calling multiple tools? You need async.

First, here's the **blocking** version:

```
import time

def say_hello():
    time.sleep(2)
    print("Hello World!")

def say_good_bye():
    time.sleep(2)
    print("GoodBye World!")

start = time.time()
say_hello()
say_good_bye()
total = time.time() - start
print(f"Total time for this version: {total:.2f} seconds")
```

This takes ~4 seconds because each function waits for the other to finish.

Blocking code, each function waits for the previous one to finish. 2 + 2 = 4 seconds wasted.

Now the **non-blocking** version:

```
import asyncio
import time

async def say_hello():
    await asyncio.sleep(2)  # Non-blocking sleep
    print("Hello World!")

async def say_good_bye():
    await asyncio.sleep(2)  # Non-blocking sleep
    print("GoodBye World!")

async def main():
    start = time.time()

    # Run both functions concurrently
    await asyncio.gather(say_hello(), say_good_bye())

    total = time.time() - start
    print(f"Total time for this version: {total:.2f} seconds")
```

```
# Run the async program
asyncio.run(main())
```

Non-blocking code, both functions run concurrently. 2 seconds total instead of 4. This is what async gives you.

The key concepts:

1. `async def`, Defines an **asynchronous function** (a coroutine)
2. `await`, Pauses execution **until an async task completes**, but lets OTHER tasks run while waiting
3. `asyncio.run()`, Starts the event loop (the engine that manages all the async tasks)
4. `asyncio.gather()`, Runs multiple async tasks **concurrently**

The common mistake that WILL bite you:

```
async def call_llm(prompt):
    # This is an async function
    response = await client.chat.completions.create(...)
    return response
```

```
# WRONG - forgetting await
result = call_llm("Hello")  # result is a coroutine object, NOT the response!

# RIGHT
result = await call_llm("Hello")  # NOW result is the actual response
```

You WILL make this mistake. When your variable is instead of an actual response, now you know why.

New Python Essentials for Agentic Systems

Here are a few more things you need in your toolkit. I'm going to teach these quickly because you'll use ALL of them in the hands-on section today.

try/except, Because Everything Can Fail

When you ask an LLM to respond in JSON, sometimes it wraps it in markdown. Sometimes it adds commentary. Sometimes it returns complete garbage. Your agent loop CANNOT crash because of this.

```
import json

llm_response = '```json\n{"tool": "calculate", "args": {"expr": "2+2"}}\n```'

# This will crash
data = json.loads(llm_response)  # JSONDecodeError!

# This won't
try:
    data = json.loads(llm_response)
except json.JSONDecodeError:
    # Strip markdown code fences and try again
    cleaned = llm_response.strip().strip('`').strip()
    if cleaned.startswith('json'):
        cleaned = cleaned[4:].strip()
    data = json.loads(cleaned)
```

Type hints + dataclasses, Contracts Between Your LLM and Your Tools

When your agent calls a tool, you need a clear contract: what fields does the LLM need to provide? What types should they be?

Dataclasses give you this:

```
from dataclasses import dataclass

@dataclass
class ToolCall:
    name: str
    arguments: dict

@dataclass
class AgentResponse:
    tool_call: ToolCall | None = None
    final_answer: str | None = None

# Now when you parse the LLM's response:
response = AgentResponse(
    tool_call=ToolCall(name="calculate", arguments={"expression": "2**10"})
)
```

```
print(response.tool_call.name)       # "calculate"
print(response.tool_call.arguments)  # {"expression": "2**10"}
```

This becomes critical in Session 6 when we move to Pydantic for full validation. For now, data classes get you started.

Decorators, Register Tools Without Maintaining a List

```
# A simple tool registry
TOOLS = {}

def tool(func):
    """Decorator that registers a function as a tool"""
    TOOLS[func.__name__] = func
    return func

@tool
def calculate(expression: str) -> str:
    """Evaluate a mathematical expression"""
    return str(eval(expression))

@tool
```

```
def get_weather(city: str) -> str:
    """Get current weather for a city"""
    return f"Weather in {city}: 28°C, partly cloudy"

# Now TOOLS automatically contains both functions
print(TOOLS)
# {'calculate': <function calculate>, 'get_weather': <function get_weather>}

# Call any tool by name
result = TOOLS["calculate"](expression="2**10")
print(result)  # "1024"
```

This pattern, registering functions by name in a dictionary, is the foundation of how every tool system works, from MCP to OpenAI function calling. Get comfortable with it.

Virtual environments, Non-negotiable

```
# Create a virtual environment
python -m venv .venv

# Activate it
source .venv/bin/activate  # macOS/Linux
```

```
.venv\Scripts\activate     # Windows

# Install dependencies
pip install google-generativeai  # or anthropic, or openai

# Or if you use uv (faster, recommended):
uv venv
source .venv/bin/activate
uv pip install google-generativeai
```

Every project. Every time. No exceptions. "It works on my machine" is not acceptable when you're building agentic systems that other agents need to interact with.

Part 2: Emergent Abilities, Why Agents Are Even Possible

Before we talk about agents, we need to answer a fundamental question: **why NOW (2024)?**

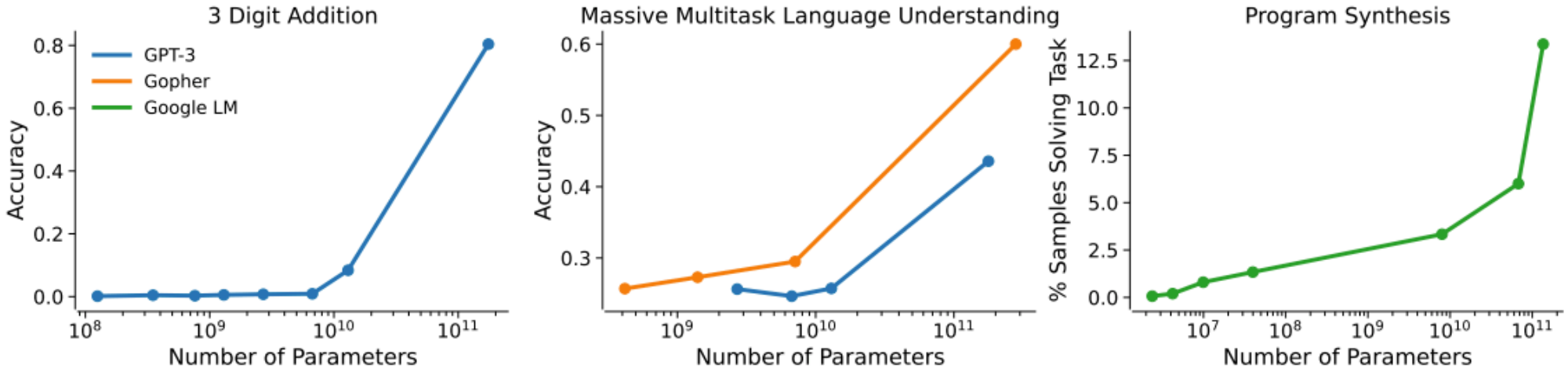
People have been trying to build autonomous AI systems for decades. Expert systems in the 80s. Rule-based bots in the 2000s. Why is 2024-2026 different?

The answer is **emergence**.

What Are Emergent Abilities?

First described in this amazing **paper**: emergent abilities refer to capabilities or behaviors that arise in LLMs as a result of their increased scale, even though they were **not explicitly programmed or anticipated** during their training or design.

These abilities are often surprising and reflect complex interactions within the neural network that become possible only at sufficient scale of parameters and data.



Emergent abilities appear suddenly at scale, not gradually. Zero capability, then sudden competence. This is a phase transition, not a smooth improvement.

These models don't gradually get better at arithmetic.

They can't do it at all, and then suddenly they CAN. That's not engineering.

That's physics. That's a phase transition, like water turning to ice.

Characteristics

- **Unpredictable:** Not explicitly encoded and often emerges without specific supervision
- **Threshold Dependent:** These abilities typically appear when the model reaches a certain threshold size or complexity, crossing a "scaling threshold"
- **Quantitative Shift:** The abilities represent a significant LEAP in performance to perform a new task, rather than a gradual improvement
- **Generalization:** Often indicates the model's ability to generalize knowledge to tasks or domains it hasn't been trained on

These abilities include

- Few-shot and Zero-shot learning
- In-context learning
- Complex reasoning
- Code generation and understanding
- Common sense and world knowledge
- Abstract thinking and metaphorical understanding

- Translation and multilingual understanding
- Social and cultural awareness
- **Tool use and API integration**, this one matters most for us

Theoretical Explanation for Emergence

- **Scale as a Driver:** Tied to scale and diversity of data
- **Representation Learning:** The model learns to represent data in increasingly rich, structured ways
- **Phase Transition:** Similar to phase change in Physics! Just like water molecules don't gradually become ice, they're liquid until they hit 0°C and then suddenly crystallize, neural networks exhibit sudden capability jumps at certain parameter thresholds.

Parameter Threshold Trends

Year	Model	Scale (Parameters)	Key Emergent Behaviors
2020	GPT-3	175B	Few-shot learning, reasoning, code generation
2022	Chinchilla	70B	Efficient scaling, emergence at smaller sizes
2023	LLaMA	7B-13B	Zero-shot, multilingual, reasoning
2023	Falcon-40B	40B	Comparable to GPT-3 in language tasks
2024	Fine-tuned LLaMA	7B	Instruction-following, reasoning via fine-tuning

The 2025-2026 Update: A New Kind of Emergence

Everything above was true in 2024. But something new has happened since then.

A new emergent ability has appeared: **internal reasoning**.

Models like Claude (with extended thinking), OpenAI's o-series (o1, o3), and Gemini (with thinking mode) have demonstrated something remarkable, they can think step by step INTERNALLY before responding.

They can:

- Break complex problems into sub-problems
- Consider multiple approaches and evaluate them
- Catch their own mistakes before outputting an answer
- Reason about uncertainty and ask for clarification

This is not just "chain of thought prompting" where we tell the model to think step by step. This is the model CHOOSING to think deeply, allocating more compute to harder problems.

Year	Model	What's New
2025	Claude 4.x (thinking)	Internal reasoning chains, extended thinking for complex problems
2025	OpenAI o-series (o1, o3)	Test-time compute scaling, spending more time on harder problems
2025	Gemini 2.x (thinking)	Multi-modal reasoning, thinking across text + images + code
2025-26	Llama 4, Qwen 3, Phi-4	Open-source reasoning models catching up fast

Why does this matter for agents?

Because the "brain" of your agent just got dramatically better at the ONE thing agents need most: **deciding what to do next**. When your agent has 10 tools available and a complex user request, a reasoning model doesn't just pattern-match, it THINKS about which tools to use, in what order, and what could go wrong.

The emergent ability that makes this entire course possible is **tool use and instruction following**. Without it, we'd be writing rule-based systems. With it, we can give an LLM a set of tools and a goal, and it will figure out how to achieve that goal. That's the magic. And it only works because of emergence.

Part 3: What Are AI Agents?

In our previous sessions, we explored transformer architecture and dove into LLM internals and SFT.

Today we're moving a step forward by examining how these foundation models can be transformed into something more dynamic and autonomous, AI agents.

Defining AI Agents:

An AI agent is a system that can perceive its environment, make decisions, and take actions to achieve specific goals. Unlike a standard LLM that simply responds to prompts, an agent has a purpose, persistence, and the ability to interact with its world.

Let me ask you three questions:

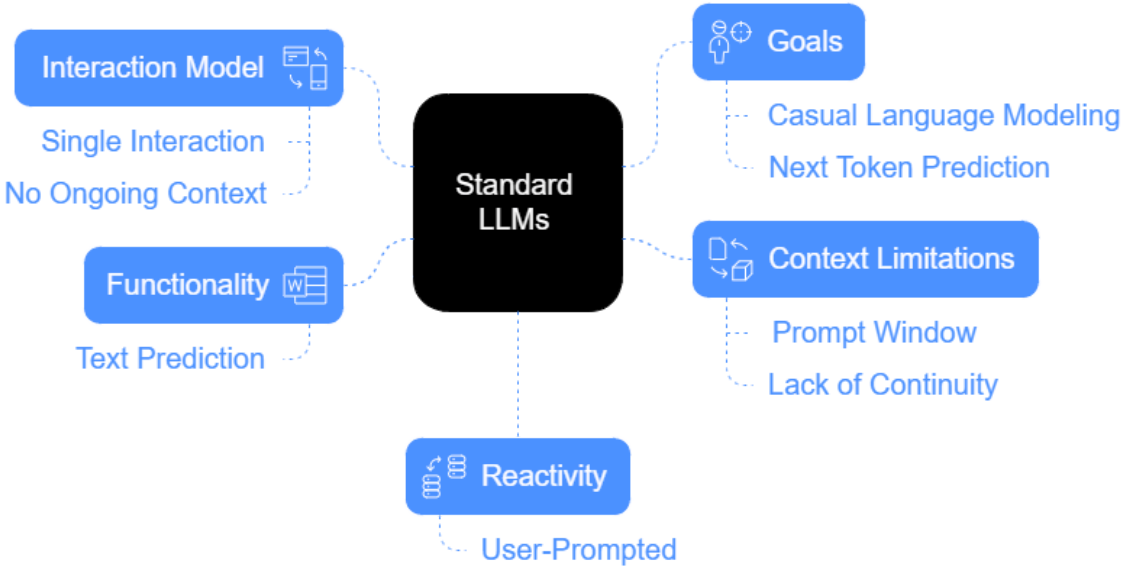
1. What is an LLM?
2. What is RAG?
3. What are AI Agents?

If your answers are fuzzy, that's OK.

Key Distinctions: LLMs vs RAG vs AI Agents

Standard LLMs:

Characteristics of Standard LLMs



Standard LLMs are sophisticated prediction engines, input in, text out, everything forgotten.

Here's what an LLM call looks like in code:

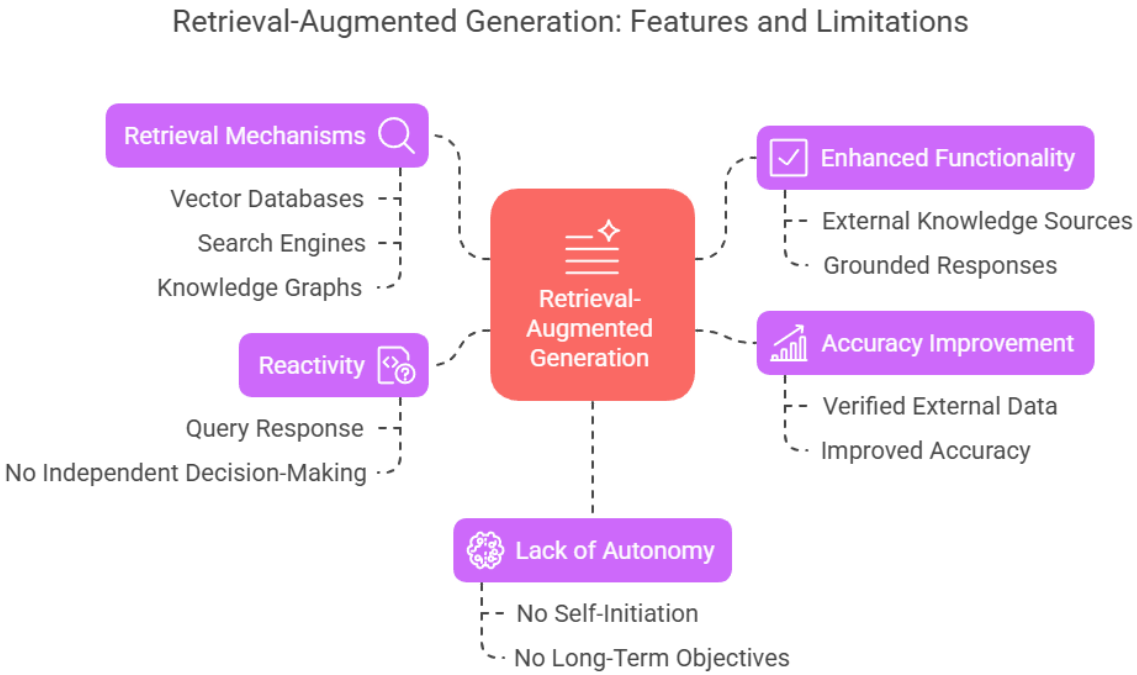
```
import google.generativeai as genai

genai.configure(api_key="YOUR_KEY")
model = genai.GenerativeModel('gemini-2.0-flash')

response = model.generate_content("What is the weather in Mumbai right now?")
```

```
print(response.text)
# "The weather in Mumbai is typically warm and humid..."
# This is a GUESS. It has no idea what the actual weather is right now.
```

- Function primarily as **sophisticated text prediction engines**
- Process **input → generate output** in a single interaction
- Have **no inherent goals** beyond *Casual Language Modeling* or next token prediction
- Are **reactive systems** that respond only when prompted
- **Forget context** outside the prompt window

The most important thing to understand: **an LLM is a function.** `text_in → text_out`. Nothing more. Everything else we build in this course is scaffolding AROUND this function.

Retrieval-Augmented Generation (RAG)



RAG makes LLMs smarter by grounding them in real data, but they still can't DO anything.

Here's what RAG looks like in code:

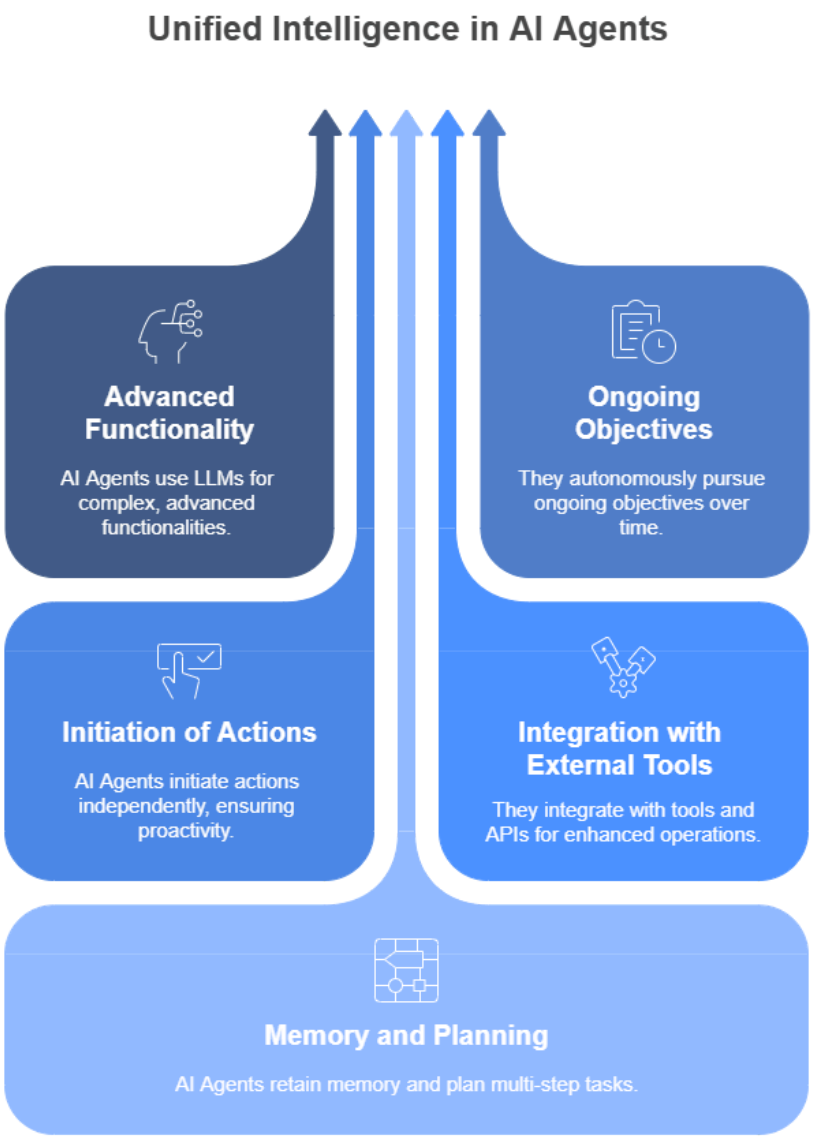
```
# Step 1: User asks a question
query = "What is our company's refund policy?"

# Step 2: RETRIEVE relevant documents from a vector database
relevant_docs = vector_db.search(query, top_k=3)
# Returns: ["Refunds are processed within 14 days...", "Full refund for items...", ...]
```

```
# Step 3: GENERATE answer using retrieved context
context = "\n".join(relevant_docs)
response = model.generate_content(
    f"Based on these documents:\n{context}\n\nAnswer: {query}"
)
print(response.text)
# Now the answer is GROUNDED in real company data, not hallucinated
```

- Extends LLMs by **fetching relevant information from external knowledge sources**
- Uses **retrieval mechanisms** (vector databases, search engines, knowledge graphs) to **ground responses** in external facts
- Improves **accuracy** and **reduces hallucinations** compared to standard LLMs
- Still **reactive**, responding only to queries, without independent decision-making
- **Lacks autonomy**, it doesn't self-initiate actions or maintain long-term objectives

RAG makes LLMs smarter but not more capable. It can answer "what is the refund policy?" but it **cannot process a refund**.

AI Agents:



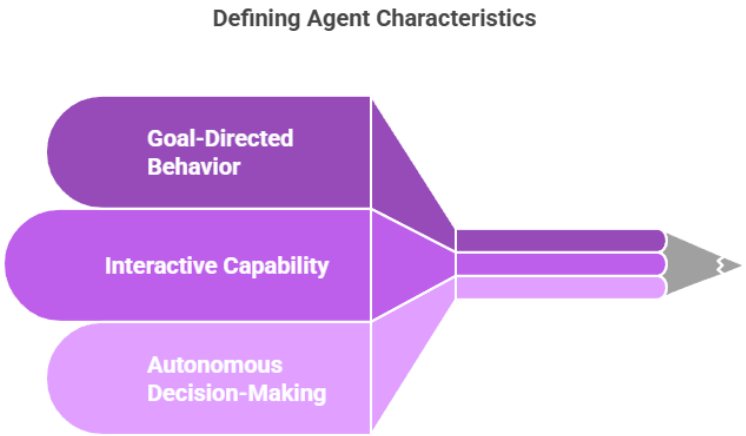
AI Agents are LLMs that can ACT, not just SPEAK. They have goals, tools, memory, and autonomy.

Here's what an agent looks like in code (simplified):

```
# An agent is an LLM in a LOOP with TOOLS
conversation_history = []
tools = {"calculate": calculate, "get_weather": get_weather, "search": search}

while not done:
    # Ask the LLM what to do next
    response = model.generate_content(system_prompt + conversation_history)

    # Did it decide to use a tool?
    if response.has_tool_call:
        tool_name = response.tool_call.name
        result = tools[tool_name](**response.tool_call.arguments)
        conversation_history.append({"tool": tool_name, "result": result})
    else:
        # It decided it has the final answer
        done = True
        print(response.final_answer)
```

- **Use LLMs as their "brain"** but extend far beyond text prediction
- Maintain **ongoing objectives** and autonomously work toward them over time
- **Can initiate actions** without human prompting
- Connect to **external tools, APIs, and real-world systems** (now using MCP, more in Session 4)
- Can **retain memory**, plan multi-step tasks, and reason about goals
- Enable **long-term decision-making**, unlike standard LLMs or RAG systems

The Three Core Pillars of Agency



Three pillars that together define what makes something an "agent" versus just a program.

What truly defines an agent comes down to three essential characteristics:

- **Goal-Directed Behavior:**

An agent has specific objectives it works to accomplish. These goals provide direction and purpose to its actions.

Example: Consider a customer service agent. Its goal isn't just to respond to messages but to **solve customer problems** with minimal escalation to human representatives.

- **Interactive Capacity:**

Agents don't exist in isolation, they interact with their environment, whether that's a digital ecosystem, databases, other software, or even the physical world through connected systems. (Now via MCP, we'll cover this in depth in Session 4)

Example: A calendar scheduling agent doesn't just suggest times for meetings, it actively checks the calendar, sends invitations, and updates appointments based on responses.

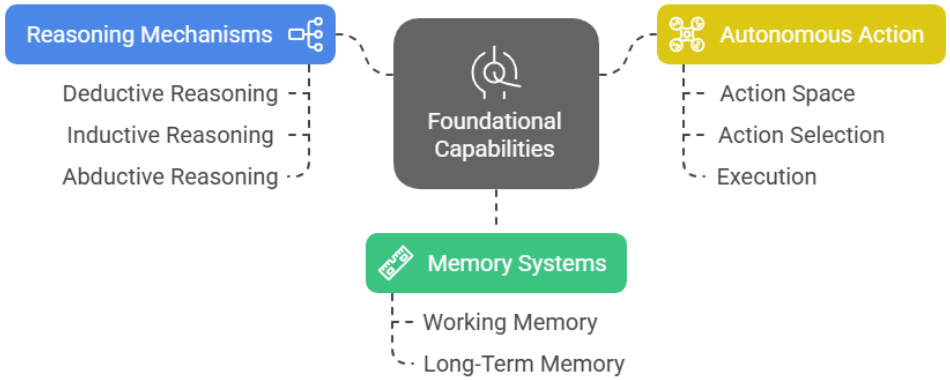
- **Autonomous Decision-Making:**

Perhaps most importantly, agents can make decisions without continuous human guidance. They can evaluate situations, choose between options, and initiate actions.

Example: A research agent might decide which source to investigate, extract relevant information, synthesize findings, and even formulate new questions to explore, all with minimal human intervention.

Core Capabilities of AI Agents

For these three pillars to function effectively, agents need several foundational capabilities:



[Image: Central node "Foundational Capabilities" connected to three boxes: "Reasoning Mechanisms" (Deductive Reasoning, Inductive Reasoning, Abductive Reasoning), "Memory Systems" (Working Memory, Long-term Memory), "Autonomous Action" (Action Space, Action Selection, Execution)]

The three foundational capabilities that make the pillars of agency work in practice.

1. Reasoning Mechanisms:

Types of Reasoning:

- **Deductive reasoning:** Drawing logical conclusions from premises

If A is true, and A → B, then B must be true!

- **Inductive reasoning:** Forming generalizations from specific observations

I've seen multiple examples of X leading to Y, so X likely causes Y.

- **Abductive reasoning:** Making best guesses on incomplete information

Given symptoms A, B, and C, the most probable cause is D.

Why is Advanced Reasoning Possible Today?

The ability of AI agents to reason effectively stems from the **emergent capabilities** of LLMs we just discussed. These capabilities arise **only beyond a certain model scale**, where the model exhibits behaviors not explicitly programmed, such as:

- *Complex pattern recognition*, identifying relationships across vast knowledge domains

- *Implicit knowledge synthesis*, drawing conclusions from scattered pieces of information
- *Meta-cognition*, assessing its own reasoning and refining its thought process

We'll go much deeper into reasoning and planning in Session 5. For today, just understand that reasoning is what allows an agent to DECIDE which tool to call and what arguments to pass.

2. Memory Systems:

- **Working Memory:**
- Maintains immediate context and recent interactions
- Enables coherent multi-turn conversations and tasks
- In our agents today, this is simply the `conversation_history` list, every message, every tool call, every result, accumulated in order
- **Long-term Memory:**
- Stores information persistently across sessions
- Enables recall of past interactions, preferences, and learned patterns

- Often implemented through vector databases or other retrieval systems
- We'll build full memory systems in Session 7

3. Autonomous Action:

- **Action Space:**
- The set of all possible actions an agent can take
- This may include API calls, database queries, tool usage, or text generation
- In our agent today: the tools dictionary defines the action space
- **Action Selection:**
- Evaluating which actions best serve the current goal
- Balancing exploration (trying new approaches) vs. exploitation (using proven methods)
- In our agent today: the LLM decides this based on the system prompt and conversation history

Part 4: State of the Art, The Agentic AI Landscape in 2026

Now before we build, let's understand the world we're building in. Things have moved FAST. Here's what you need to know.

The Protocol Stack Revolution

The biggest structural change in the agentic AI world is the emergence of three standardized protocols. Think of them like the TCP/IP stack for AI:

Layer	Protocol	What It Connects	Status in 2026
Tools	**MCP** (Anthropic)	Agent ↔ Tools/APIs	Mature, industry standard. Claude, Cursor, VS Code, 1000+ servers. You'll learn this in **Session 4**.
Agents	**A2A** (Google)	Agent ↔ Agent	Production-ready. 50+ partners, Linux Foundation governance. Agent Cards for discovery. You'll learn this in **Session 13**.

Layer	Protocol	What It Connects	Status in 2026
UI	**A2UI / AG-UI**	Agent ↔ User Interface	Emerging. Agents generating their own UIs. A2UI (Google, declarative), AG-UI (CopilotKit/Oracle/Microsoft, streaming). You'll learn this in **Session 14**.

In V2, we focused heavily on MCP. In V3, you'll master all three layers. By the end, your agent will be able to use tools (MCP), coordinate with other students' agents (A2A), and generate dynamic UIs for users (A2UI/AG-UI).

Let's look at some mind-blowing MCP examples that show what's already possible:

- **Blender MCP**

  ▷

  , An AI agent that can CREATE 3D models in Blender just by talking to it
- **Cursor Talk to Figma MCP**, Design in Figma by describing what you want

- **List of MCP Servers**, Official list maintained by Anthropic
- **Awesome MCP Servers**, Community-curated list

The know-how on Agentic AI has moved from "knowing the framework" to "building protocol-based tools." MCP is still young and will see a lot of evolution, but the pattern is clear: **tools are becoming standardized, and agents that speak the protocol win.**

The Autonomy Frontier

The other big shift is how autonomous agents have become. Let me show you the spectrum:

Level 1: Chat Assistants (2023)

You type, it responds. ChatGPT, Claude chat. One turn at a time. No tools, no memory.

Level 2: Tool-Using Agents (2024)

The agent can call functions. Gemini can search the web. Claude can use tools. But still reactive, you ask, it acts.

Level 3: Autonomous Agents (2025-2026)

This is where it gets interesting:

- **Claude Code / Cursor / Windsurf**, Coding agents that read your entire codebase, understand the architecture, write code, run tests, fix failures, and commit. They don't just answer questions about code, they WRITE code.

- **Anthropic Computer Use / OpenAI Operator**, Agents that can see your screen, move the mouse, click buttons, fill forms. They operate your computer like a human would.
- **Karpathy's autoresearch**, Perhaps the purest example of agent autonomy. Three files. The agent modifies code, trains for exactly 5 minutes, checks if the result improved, keeps or discards (via git commit/reset), and repeats. ~100 experiments overnight while the human sleeps. The key principle: **"Human programs the program, not the code."** You don't touch Python. You iterate on `program.md`, the markdown instructions that shape agent behavior.
- **OpenClaw / Arcturus**, Full agentic platforms with channels (WhatsApp, Telegram, Slack, Discord), voice, browser automation, container isolation, DAG execution engines. These are the systems you'll learn to build in this course.

Level 4: Multi-Agent Systems (2026)

Multiple autonomous agents coordinating via A2A protocol. Your agent discovers other agents, delegates subtasks, synthesizes results. This is Session 13.

What "Best in the World" Means by Session 19

By the time you finish this course, you should be able to:

1. Build a multi-agent DAG executor from scratch
2. Wire it to any channel (WhatsApp, Slack, Discord, voice) in a day
3. Give it browser autonomy and computer use
4. Give it sandboxed code execution in containers
5. Make it remember, learn preferences, and recall past workflows
6. Deploy with observability and cost controls
7. Evaluate and benchmark against GAIA-level tasks
8. Expose it as a platform API that other agents call (A2A)
9. Have it generate dynamic UIs for users (A2UI/AG-UI)
10. Have it operate autonomously on event streams without human prompting

That's the bar. Today we start with #1, the simplest possible version of an agent. One agent, one loop, a few tools. But this is the seed from which everything else grows.

Part 5: Hands-On, Let's Build Your First Agent

Let's build.

- First, let's play with the LLM
- Then give it a system prompt that makes it behave like an agent
- Define tools as Python functions
- Check if the LLM responds properly with tool calls
- Parse its results
- Call the functions
- Put everything in a loop

Step 1: Talk to an LLM

Let's start simple. Just call the API and see what comes back.

```
import google.generativeai as genai
import os

genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel('gemini-3.0-flash')
```

```
response = model.generate_content("What is 2 raised to the power of 10?")
print(response.text)
```

Run this. You'll get something like "2^10 = 1024." Great. The LLM knows math (sort of).

Now try:

```
response = model.generate_content("What is the current temperature in Mumbai?")
print(response.text)
```

It'll give you a plausible-sounding answer. But it's WRONG. It has no idea what the actual temperature is right now. It's making it up based on patterns. This is the gap we need to fill.

Step 2: Give It a System Prompt That Makes It an Agent

We need to tell the LLM: "You have tools. When you need real-world data or calculations, use them. Respond in a structured format so I can parse your answer."

```
system_prompt = """You are a helpful AI agent that can use tools to answer questions.

You have access to the following tools:

1. calculate(expression: str) -> str
   Evaluate a mathematical expression. Example: calculate("2**10")

2. get_weather(city: str) -> str
   Get the current weather for a city. Example: get_weather("Mumbai")

3. search_notes(query: str) -> str
   Search through user's notes. Example: search_notes("meeting agenda")

You must respond in ONE of these two JSON formats:

If you need to use a tool:
{"tool_name": "<name>", "tool_arguments": {"<arg_name>": "<value>"}}

If you have the final answer:
{"answer": "<your final answer>"}

IMPORTANT: Respond with ONLY the JSON. No other text. No markdown. No code fences.
"""
```

Notice what we're doing: we're converting an LLM into an agent by constraining its output format. The LLM doesn't "know" it's an agent. We're giving it a role, tools, and a structured output format. The intelligence comes from the model. The structure

comes from us.

Step 3: Define Tools as Python Functions

```
import json
import math

def calculate(expression: str) -> str:
    """Evaluate a mathematical expression safely"""
    try:
        # Using a safer eval with limited scope
        allowed_names = {"math": math, "abs": abs, "round": round}
        result = eval(expression, {"__builtins__": {}}, allowed_names)
        return json.dumps({"result": str(result)})
    except Exception as e:
        return json.dumps({"error": str(e)})

def get_weather(city: str) -> str:
    """Get current weather for a city (simulated for now)"""
    # In a real agent, this would call a weather API
    weather_data = {
        "Mumbai": "32°C, Humid, Partly Cloudy",
        "Delhi": "28°C, Clear Sky",
        "London": "15°C, Rainy",
        "New York": "22°C, Sunny",
```

```
    }
    weather = weather_data.get(city, f"Weather data not available for {city}")
    return json.dumps({"weather": weather})

def search_notes(query: str) -> str:
    """Search through user's notes (simulated)"""
    notes = [
        {"title": "Meeting Agenda", "content": "Discuss Q3 targets, review agent architecture"},
        {"title": "Shopping List", "content": "Milk, eggs, bread, coffee"},
        {"title": "Project Ideas", "content": "Build a stock monitoring agent, voice-based assistant"},
    ]
    results = [n for n in notes if query.lower() in n["title"].lower() or query.lower() in n["content"].lower()]
    return json.dumps({"results": results if results else "No notes found matching your query"})

# Register tools in a dictionary
tools = {
    "calculate": calculate,
    "get_weather": get_weather,
    "search_notes": search_notes,
}
```

See that `tools` dictionary at the bottom? That's our tool registry. When the LLM says "call calculate with expression 2**10", we look up `tools["calculate"]` and call it. Simple.

Step 4: Check If the LLM Responds Properly

Let's test if our system prompt actually makes the LLM respond in the right format:

```
user_message = "What is the weather in Mumbai?"

response = model.generate_content(
    f"{system_prompt}\n\nUser: {user_message}"
)

print("Raw LLM response:")
print(response.text)
print()

# Try to parse it
try:
    parsed = json.loads(response.text)
    print("Parsed successfully!")
    print(json.dumps(parsed, indent=2))
except json.JSONDecodeError as e:
    print(f"Failed to parse: {e}")
    print("This is why we need robust parsing!")
```

Run this a few times. Sometimes the LLM will be perfect. Sometimes it'll add commentary or wrap it in markdown code fences. This is REALITY. LLMs are not deterministic. Your code must handle the mess.

Step 5: Robust Parsing

```
def parse_llm_response(response_text: str) -> dict:
    """Parse the LLM's response, handling common formatting issues"""
    text = response_text.strip()

    # Remove markdown code fences if present
    if text.startswith("```"):
        lines = text.split("\n")
        # Remove first and last lines (the fences)
        lines = lines[1:-1] if lines[-1].strip() == "```" else lines[1:]
        text = "\n".join(lines).strip()
        # Remove language identifier (e.g., "json")
        if text.startswith("json"):
            text = text[4:].strip()

    try:
        return json.loads(text)
    except json.JSONDecodeError:
        # Last resort: try to find JSON in the response
```

```
        import re
        json_match = re.search(r'\{.*\}', text, re.DOTALL)
        if json_match:
            return json.loads(json_match.group())
        raise ValueError(f"Could not parse LLM response: {text}")
```

This is a pattern you'll use throughout the course. LLMs are messy. Your parsing code must be resilient.

Step 6: Call the Tool and Feed Results Back

Now the key insight: after we get a tool result, we need to send it BACK to the LLM so it can decide what to do next. This is where the "loop" starts to form.

```
user_message = "What's the weather in Mumbai and is it hotter than 30 degrees?"

# Build conversation history
conversation = f"{system_prompt}\n\nUser: {user_message}"

# First LLM call
```

```
response = model.generate_content(conversation)
parsed = parse_llm_response(response.text)
print(f"LLM says: {json.dumps(parsed, indent=2)}")

# It should ask for weather tool
if "tool_name" in parsed:
    tool_name = parsed["tool_name"]
    tool_args = parsed["tool_arguments"]

    # Call the tool
    tool_result = tools[tool_name](**tool_args)
    print(f"\nTool '{tool_name}' returned: {tool_result}")

    # Feed the result back to the LLM
    conversation += f"\n\nAssistant: {response.text}"
    conversation += f"\n\nTool Result ({tool_name}): {tool_result}"
    conversation += "\n\nBased on the tool result, please provide your response."

    # Second LLM call
    response2 = model.generate_content(conversation)
    parsed2 = parse_llm_response(response2.text)
    print(f"\nLLM final response: {json.dumps(parsed2, indent=2)}")
```

Look at what just happened. The LLM couldn't answer the question on its own. It asked for a tool. We called the tool. We gave the result back. The LLM used that real data to give a proper answer. **That's an agent.**

Step 7: Put Everything in a Loop

Now let's put it all together, the full agent loop:

```
import google.generativeai as genai
import json
import re
import math
import os

# --- Configuration ---
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))
model = genai.GenerativeModel('gemini-3.0-flash')

# --- System Prompt ---
system_prompt = """You are a helpful AI agent that can use tools to answer questions accurately.

You have access to the following tools:

1. calculate(expression: str) -> str
   Evaluate a mathematical expression. Example: calculate("2**10")

2. get_weather(city: str) -> str
   Get the current weather for a city. Example: get_weather("Mumbai")
```

```
3. search_notes(query: str) -> str
   Search through user's notes. Example: search_notes("meeting agenda")

You must respond in ONE of these two JSON formats:

If you need to use a tool:
{"tool_name": "<name>", "tool_arguments": {"<arg_name>": "<value>"}}

If you have the final answer:
{"answer": "<your final answer>"}

IMPORTANT:
- Respond with ONLY the JSON. No other text.
- Use tools when you need real data or calculations.
- After receiving a tool result, either use another tool or provide your final answer.
"""

# --- Tools ---
def calculate(expression: str) -> str:
    try:
        allowed = {"math": math, "abs": abs, "round": round, "pow": pow}
        result = eval(expression, {"__builtins__": {}}, allowed)
        return json.dumps({"result": str(result)})
    except Exception as e:
        return json.dumps({"error": str(e)})

def get_weather(city: str) -> str:
    weather_data = {
        "Mumbai": "32°C, Humid, Partly Cloudy",
        "Delhi": "28°C, Clear Sky",
```

```
        "London": "15°C, Rainy",
        "New York": "22°C, Sunny",
        "Tokyo": "26°C, Windy",
    }
    weather = weather_data.get(city, f"Weather data not available for {city}")
    return json.dumps({"weather": weather})

def search_notes(query: str) -> str:
    notes = [
        {"title": "Meeting Agenda", "content": "Discuss Q3 targets, review agent architecture"},
        {"title": "Shopping List", "content": "Milk, eggs, bread, coffee"},
        {"title": "Project Ideas", "content": "Build a stock monitoring agent, voice assistant"},
    ]
    results = [n for n in notes if query.lower() in n["title"].lower() or query.lower() in n["content"].lower()]
    return json.dumps({"results": results if results else "No notes found"})

tools = {
    "calculate": calculate,
    "get_weather": get_weather,
    "search_notes": search_notes,
}

# --- Response Parser ---
def parse_llm_response(text: str) -> dict:
    text = text.strip()
    if text.startswith("```"):
        lines = text.split("\n")
        lines = lines[1:-1] if lines[-1].strip() == "```" else lines[1:]
        text = "\n".join(lines).strip()
        if text.startswith("json"):
```

```
            text = text[4:].strip()
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        json_match = re.search(r'\{.*\}', text, re.DOTALL)
        if json_match:
            return json.loads(json_match.group())
        raise ValueError(f"Could not parse: {text}")

# --- The Agent Loop ---
def run_agent(user_query: str, max_iterations: int = 5):
    print(f"\n{'='*60}")
    print(f"User: {user_query}")
    print(f"{'='*60}")

    # Build initial conversation
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_query},
    ]

    for iteration in range(max_iterations):
        print(f"\n--- Iteration {iteration + 1} ---")

        # Build the prompt from message history
        prompt = ""
        for msg in messages:
            if msg["role"] == "system":
                prompt += msg["content"] + "\n\n"
            elif msg["role"] == "user":
```

```
            prompt += f"User: {msg['content']}\n\n"
        elif msg["role"] == "assistant":
            prompt += f"Assistant: {msg['content']}\n\n"
        elif msg["role"] == "tool":
            prompt += f"Tool Result: {msg['content']}\n\n"

    # Call the LLM
    response = model.generate_content(prompt)
    response_text = response.text
    print(f"LLM Response: {response_text}")

    # Parse the response
    try:
        parsed = parse_llm_response(response_text)
    except (ValueError, json.JSONDecodeError) as e:
        print(f"Parse error: {e}")
        print("Asking LLM to try again...")
        messages.append({"role": "assistant", "content": response_text})
        messages.append({"role": "user", "content": "Please respond with valid JSON only."})
        continue

    # Check if it's a final answer
    if "answer" in parsed:
        print(f"\n{'='*60}")
        print(f"Agent Answer: {parsed['answer']}")
        print(f"{'='*60}")
        return parsed["answer"]

    # It's a tool call
    if "tool_name" in parsed:
```

```
            tool_name = parsed["tool_name"]
            tool_args = parsed.get("tool_arguments", {})

            print(f"Calling tool: {tool_name}({tool_args})")

            if tool_name not in tools:
                print(f"Unknown tool: {tool_name}")
                messages.append({"role": "assistant", "content": response_text})
                messages.append({"role": "tool", "content": json.dumps({"error": f"Unknown tool: {tool_name}"})})
                continue

            # Execute the tool
            tool_result = tools[tool_name](**tool_args)
            print(f"Tool Result: {tool_result}")

            # Add to conversation history
            messages.append({"role": "assistant", "content": response_text})
            messages.append({"role": "tool", "content": tool_result})

    print("\nMax iterations reached. Agent could not complete the task.")
    return None


# --- Run it! ---
if __name__ == "__main__":
    # Test 1: Simple tool call
    run_agent("What is the weather in Mumbai?")

    # Test 2: Calculation
    run_agent("What is 2 raised to the power of 10, plus the square root of 144?")
```

```
# Test 3: Multi-step reasoning
run_agent("Search my notes for project ideas, then tell me the weather in Tokyo so I can decide if I should work fro

# Test 4: Something that needs multiple tools
run_agent("Calculate the sum of exponential values of the first 6 Fibonacci numbers")
```

Let's look at what's happening visually:

```
User Query: "What's the weather in Mumbai and is it hotter than 30°C?"
    |
    ▼
┌──────────────────────────────────────────┐
| LLM (system prompt + conversation history)   |
|                                          |
| Thinks: "I need real weather data for Mumbai"|
| Returns: {"tool_name": "get_weather",        |
|           "tool_arguments": {"city":"Mumbai"}}|
└──────────────────────────────────────────┘
               |
               |
               ▼
┌──────────────────────────────────────────┐
```

```
│ Execute: get_weather("Mumbai")                  │
│ Result: {"weather": "32°C, Humid, Cloudy"}      │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
          Append to history
                  │
                  ▼
┌─────────────────────────────────────────────────┐
│ LLM (system prompt + UPDATED history)           │
│                                                 │
│ Thinks: "32°C > 30°C, so yes, it's hotter"      │
│ Returns: {"answer": "The weather in Mumbai      │
│  is 32°C. Yes, it is hotter than 30°C."}        │
└─────────────────────────────────────────────────┘
                  │
                  │
                  ▼
            Final Answer!
```

This is it. This is the pattern. Everything else in this course, MCP, multi-agent systems, DAG execution, browser agents, computer use, is built on top of this loop. The tools get more sophisticated. The planning gets smarter. The memory gets persistent.

But the core loop is always:

LLM decides → Tool executes → Result feeds back → LLM decides again

Burn this into your brain. Draw it from memory. If you understand this loop deeply, you understand agents.

Part 6: Why You Must Code Without AI

Before we close, I need to say something important.

You just built an agent. Congratulations. But I want you to ask yourself honestly: **could you have written that code without Copilot? Without ChatGPT?**

Here's the uncomfortable truth. There are people right now who can get Claude to write them an entire agent framework. They can prompt their way to a working demo. But when it breaks, when the LLM returns unexpected JSON, when the tool throws an exception they didn't anticipate, when the loop runs forever, they're stuck. They can't read the stack trace. They can't reason about the control flow. They can't set a breakpoint and step through the logic.

The agents you're building in this course will generate code. Session 17 is literally about coding agents. If you can't evaluate the code your agent produces, you can't trust your agent. You're building a system you don't understand, and that's dangerous.

Here's my ask: for the rest of this course, there will be moments where I'll ask you to code without AI assistance. Not because AI tools are bad, they're amazing. But because you need to know that the floor beneath you is solid. That if all the AI tools disappeared tomorrow, you could still build.

The best agent developers I know don't use AI to write code they don't understand. They use AI to write code FASTER that they ALREADY understand. There's a massive difference.

Assignment

Make a simple Agentic AI Chrome Plugin!

- In the first assignment, you made a simple Chrome Plugin
- In the second assignment, you added LLM to it

- In this, third assignment, you need to make a **Chrome Agentic AI Plugin**!
- Now the expectation is:
- You ask your LLM something it cannot do (or would hallucinate, or would internally call its own Agent, which Gemini can do!), for example:
- **Calculations:** Calculate the sum of exponential values of the first 6 Fibonacci Numbers
- **External Tool/Framework Use:** Find the top OTT series this week and send them to me on Telegram/Email
- **Continuous Monitoring:** Track the price of a Stock (or Temperature) and update me as soon as it crosses X value
- **Multi-step Research:** Find the news about a particular stock and link it with its price changes (e.g. search news about Ola in the last 1 month and news date, then see how the stock moved on those dates, and then link this data)
- So to achieve this assignment you must:
- Call your LLM multiple times:
- Query→LLM Response→Tool Call:Tool Result→Query→LLM Response→Tool Call:Tool Result→Query→LLM Response→Result
- Each time your Query stores "ALL" past interaction
- **Query1**→LLM Response→Tool Call:Tool Result→**Query2**→LLM Response→Tool Call:Tool Result→**Query3**→LLM Response→Result
- **NEW:** Your extension must display the agent's reasoning chain, show each tool call and result, not just the final answer
- **NEW:** You must have at least 3 custom tool functions (pick functions relevant to YOUR use case, each student should have different tools!)

- Keep things very simple.
- Submission: A YouTube video that shows how your Agent works on your Chrome Plugin AND copy-paste your LLM logs.
- You can find the class code **here** for your practise.
- Please remember "Chrome Plugin" is a suggestion, if you want to make something complex its totally fine.